Listing 8.3 `Movie` object type

```
class Movie {
    constructor(id, name, year) {
        this.id = id;
        this.name = name;
        this.year = year;
    }
}

module.exports = Movie;
```

The repository example in listing 8.4 sets the map value, but this is where the database code would go. The movie repository generates the `id` value and passes the event data to the data source.

Listing 8.4 Movie repository

```
class MovieRepository {

    constructor() {
        this.movies = new Map([]);
    }

    async insert(movie) {
        const id = 100;     #1
        return this.movies.set(id, movie.name, movie.year);
    }
}

module.exports = new MovieRepository();
```

#1 Updates the database id

Once the movie handler is defined, we need to set up the Kafka service, which contains the unique identifier for the client with the client ID and the topic subscription “movies” in this example. Upon receiving an event, handle each message by passing the message from the event as a JSON object.